

Labwork 3

Trinh Thanh Vinh - 2540110

May 23, 2026

1 Introduction

This report presents the implementation of a simple neural network using the backpropagation algorithm in Python. The neural network was implemented from scratch without using libraries such as TensorFlow or PyTorch. The objective of this experiment is to train the neural network to solve the XOR classification problem.

2 Neural Network Structure

The neural network consists of input layer, hidden layer, and output layer. The network structure is loaded from a file named `struc.txt`. For the XOR experiment, the structure contains 3 layers:

- 2 input neurons
- 3 hidden neurons
- 1 output neuron

3 Feedforward Process

Each neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- w_i are the weights
- x_i are the inputs
- b is the bias

The sigmoid activation function is used:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The neuron output becomes:

$$a = \sigma(z)$$

4 Backpropagation Implementation

Backpropagation is used to compute gradients for updating weights and biases.

4.1 Output Layer Delta

For the output neuron, the delta is computed as:

$$\delta = \hat{y} - y$$

where:

- $\hat{y}$ is the predicted output
- y is the true label

In the implementation:

```
s = neuron.sigmoid(-neuron.last_z)
neuron.delta = -y_true[j] + (1 - s)
```

Since:

$$1 - \sigma(-z) = \sigma(z)$$

the equation becomes:

$$\delta = \sigma(z) - y$$

4.2 Hidden Layer Delta

For hidden neurons:

$$\delta_j = \sigma(z_j)(1 - \sigma(z_j)) \sum_k w_{jk} \delta_k$$

where:

- w_{jk} is the weight connecting hidden neuron j to neuron k
- δ_k is the delta from the next layer

Implementation:

```
grad_sum += next_neuron.weights[j] * next_neuron.delta
neuron.delta = (1 - s) * s * grad_sum
```

4.3 Gradient Descent Update

Weights and biases are updated using gradient descent:

$$w = w - \eta \delta x$$

$$b = b - \eta \delta$$

where:

- η is the learning rate

Implementation:

```
neuron.weights[k] =
    neuron.weights[k]
    - 1 * neuron.delta * neuron.last_input[k]

neuron.bias =
    neuron.bias - 1 * neuron.delta
```

5 Loss Function

The binary cross-entropy loss function is used:

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i z_i - \log(1 + e^{z_i}))$$

where:

- N is the number of samples
- z_i is the neuron output before activation

6 XOR Dataset Experiment

6.1 Dataset

The XOR dataset:

$$(0, 0) \rightarrow 0$$

$$(0, 1) \rightarrow 1$$

$$(1, 0) \rightarrow 1$$

$$(1, 1) \rightarrow 0$$

6.2 Training Configuration

- Learning rate: 0.5
- Maximum epochs: 50000
- Activation function: Sigmoid

7 Results

After 50000 epochs, the network produced the following predictions:

[0, 0] -> predicted: 8.051108211763272e-05
expected: 0

[0, 1] -> predicted: 0.999916280093545
expected: 1

[1, 0] -> predicted: 0.9999180963066683
expected: 1

[1, 1] -> predicted: 0.0001239938381239292
expected: 0

8 Analysis

The neural network successfully learned the XOR function, with the predictions being extremely close to the expected outputs. This demonstrates that the backpropagation algorithm was implemented correctly, allowing the network to adjust its weights and biases to minimize the loss function effectively. The hidden layer allows the network to capture the non-linear relationships in the XOR dataset, which is not linearly separable. The gradient descent successfully minimized the loss function, leading to accurate predictions after training.